

Retail Data Warehouse ETL

End-to-End ETL Workflow & Architecture

1. Purpose

This document explains the end-to-end data pipeline implemented for the Retail Data Warehouse project.

The pipeline extracts data from the operational MySQL database, loads it into staging, performs incremental and change-aware processing, applies data-quality validations, loads dimensions and fact tables, performs reconciliation, and exposes the data through reporting views and a dashboard.

The overall objective is to build a reusable and production-oriented ETL framework that supports:

- Full initial loads
- Incremental loads
- Watermark-based extraction
- Change detection using row hashes
- SCD Type 2 dimensions
- Fact table loading
- Data-quality validation
- Source-to-target reconciliation
- Automated testing
- Reporting and dashboards

2. Source and Target Architecture

We have two MySQL databases.

Source Database

```
intern_sql_training
```

This represents the operational/source system.

The source contains eight tables:

```
categories
customers
departments
employees
order_items
orders
```

payments
products

Target Database

retail_dw

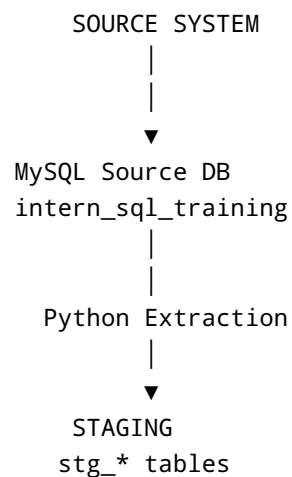
This represents the analytical data warehouse.

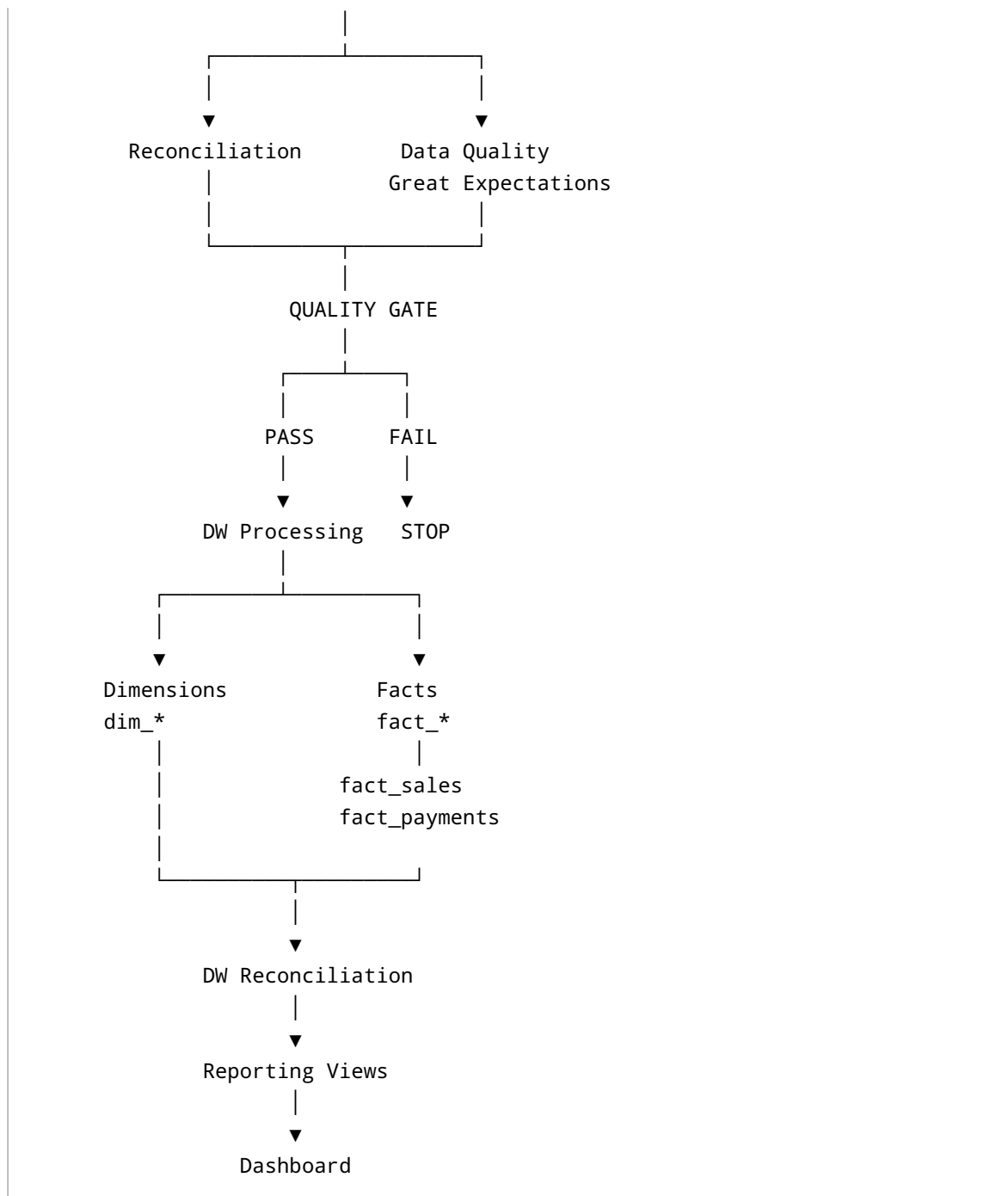
The target contains:

stg_*	→ Staging tables
dim_*	→ Dimension tables
fact_sales	→ Sales fact
fact_payments	→ Payment fact
etl_batch_control	→ ETL execution metadata
etl_watermark	→ Incremental processing state
etl_record_hash	→ Change detection metadata
vw_*	→ Reporting views

3. High-Level Data Flow

The complete architecture is:





4. Why Staging Exists

The staging layer acts as a buffer between the operational source system and the analytical warehouse.

We do not directly load:

Source → Dimension/Fact

Instead:

Source → Staging → Warehouse

This provides several advantages:

- Source and target systems remain decoupled.
- Raw source data can be inspected before transformation.
- Data-quality checks can be performed.
- Reconciliation can be performed.
- Failed warehouse loads do not require re-querying the source.
- ETL debugging becomes easier.
- Incremental processing can be controlled independently.

5. Step 1 – Initial Full Load

The first execution performs a full extraction.

For example:

```
customers
  ↓
stg_customers

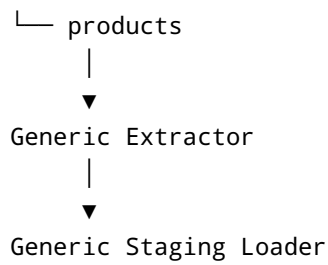
products
  ↓
stg_products

orders
  ↓
stg_orders
```

The generic Python framework reads the table configuration and processes all eight tables.

Conceptually:

```
TABLE_CONFIG
├── categories
├── customers
├── departments
├── employees
├── order_items
├── orders
├── payments
```



The advantage of using configuration-driven processing is that the same extraction and loading logic can process multiple tables.

6. ETL Batch Control

Every ETL execution receives a batch ID.

Example:

```
Batch ID: 25
```

The batch is recorded in:

```
etl_batch_control
```

This table provides ETL audit information.

Typical information includes:

```
batch_id
pipeline_name
start_time
end_time
status
records_processed
error_message
```

Example:

```
batch_id    : 25
pipeline    : SOURCE_TO_STAGING
status      : SUCCESS
records     : 271
```

If an error occurs:

```
batch_id    : 26
status      : FAILED
error       : Data quality validation failed
```

This allows us to answer:

Which ETL run loaded this data?

7. Step 2 – Incremental Processing

After the initial load, we don't want to extract every record from the source every time.

For example, suppose the source contains:

```
customers = 20
```

The first run loads all 20 records.

Later, customer 21 is inserted.

Instead of extracting:

```
1 → 21
```

we extract only:

```
21
```

This is the purpose of incremental loading.

8. Watermark-Based Incremental Extraction

The pipeline maintains the last successfully processed value in:

```
etl_watermark
```

For example:

```
table_name    watermark
-----
```

customers	20
orders	45
products	30

When the next run starts:

```
customer_id > 20
```

is used to identify new records.

If customer 21 exists:

```
20 → 21
```

the extractor retrieves customer 21.

The watermark is then advanced:

```
Old watermark: 20  
New watermark: 21
```

9. Why Watermarks Are Important

Watermarks prevent us from repeatedly processing the same data.

Without a watermark:

```
Run 1 → 20 records  
Run 2 → 21 records  
Run 3 → 21 records  
Run 4 → 21 records
```

With a watermark:

```
Run 1 → 20 records  
  
Run 2 → 1 new record  
  
Run 3 → 0 records  
  
Run 4 → 0 records
```

This significantly reduces unnecessary source reads and processing.

10. Important Limitation of Watermarks

A watermark is excellent for detecting **new records**, but it does not by itself detect updates.

Example:

```
customer_id = 22  
city = Hyderabad
```

Later:

```
customer_id = 22  
city = Secunderabad
```

The customer ID is still:

```
22
```

Therefore:

```
customer_id > watermark
```

will not detect the update.

This is why we introduced **change detection**.

11. Step 3 – Hash-Based Change Detection

For each business record, we generate a hash from its relevant attributes.

Conceptually:

```
customer_id  
customer_name  
email  
city  
state  
|
```

▼ Row Hash

Example:

Customer 22
Before: city = Hyderabad
Hash: ABC123...

After:

city = Secunderabad
Hash: XYZ789...

Because:

ABC123 != XYZ789

we know that the record changed.

12. Change Classification

Every source record can be classified as:

NEW
CHANGED
UNCHANGED

NEW

Record does not exist in the target metadata.

Source → NEW

CHANGED

Record exists but its hash is different.

Source hash != Stored hash

UNCHANGED

Record exists and the hash is identical.

Source hash == Stored hash

Example:

```
Customer 22  
  
NEW          : 0  
CHANGED      : 1  
UNCHANGED    : 21
```

13. Why We Need Both Watermark and Hash Detection

These two mechanisms solve different problems.

Watermark

Detects:

NEW RECORDS

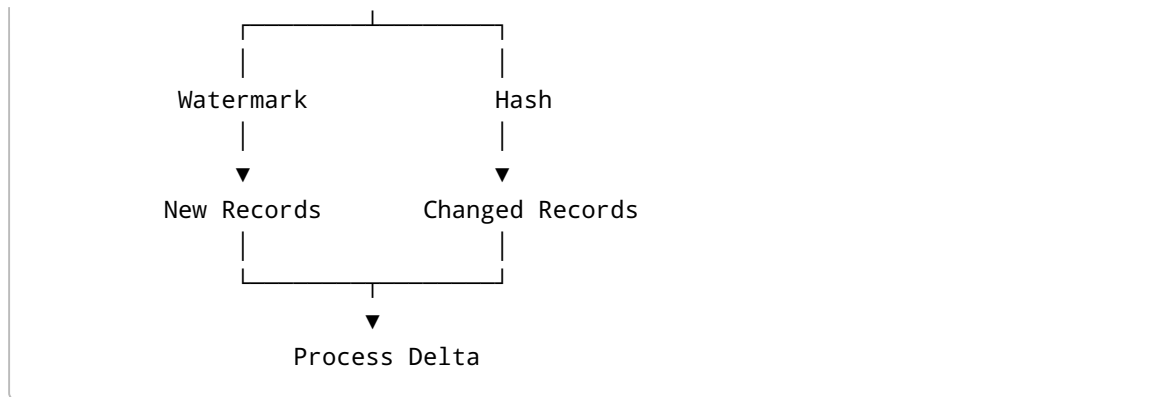
Hash

Detects:

CHANGED RECORDS

Together:

Incremental Processing



This is the core of our change-aware ETL.

14. Step 4 – Data Quality Gate

After staging, the pipeline validates the data before loading the warehouse.

Great Expectations is used as the data-quality framework.

Typical checks include:

Customers

```
customer_id NOT NULL
customer_id UNIQUE
customer_name NOT NULL
```

Products

```
product_id NOT NULL
product_id UNIQUE
product_name NOT NULL
unit_price >= 0
```

Orders

```
order_id NOT NULL
order_id UNIQUE
customer_id NOT NULL
order_date NOT NULL
```

Order Items

```
order_item_id UNIQUE
order_id NOT NULL
product_id NOT NULL
quantity > 0
unit_price >= 0
```

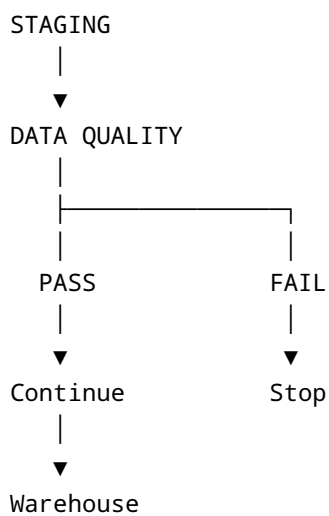
Payments

```
payment_id UNIQUE
order_id NOT NULL
amount >= 0
```

15. Data Quality as a Pipeline Gate

The pipeline should not blindly continue when data quality fails.

The logic is:



For example:

```
unit_price = -100
```

would cause:

```
Data Quality: FAIL
```

and the warehouse load should not continue.

16. Step 5 – Dimension Loading

After the data passes quality checks, we load dimensions.

The project contains dimensions such as:

```
dim_customer
dim_product
dim_category
dim_department
dim_employee
dim_date
```

Dimensions contain descriptive information used to analyze facts.

For example:

```
dim_customer

customer_sk
customer_id
customer_name
city
state
effective_from
effective_to
is_current
```

17. SCD Type 2

Customer and product changes need historical tracking.

For example:

Initial record:

```
customer_id = 22
city        = Hyderabad
is_current  = 1
```

Later the customer moves:

```
city = Secunderabad
```

We do not overwrite the old record.

Instead:

customer_id	city	is_current
22	Hyderabad	0
22	Secunderabad	1

The old version is preserved.

This is **Slowly Changing Dimension Type 2**.

18. SCD Type 2 Processing

When a changed record is detected:

```
graph TD
    Source --> Hash[Hash comparison]
    Hash --> CHANGED
    CHANGED --> Close[Close existing dimension record]
    Close --> Insert[Insert new version]
    Insert --> Mark[Mark new version as current]
```

The warehouse therefore maintains historical information.

19. Step 6 – Fact Sales

The sales fact is created from:

```
orders
+
order_items
```

The grain of `fact_sales` is:

One row per order item.

For example:

```
Order 100
|
├── Laptop
├── Mouse
└── Keyboard
```

creates three fact rows.

The sales amount is calculated as:

```
quantity × unit_price
```

Example:

```
quantity    = 2
unit_price  = ₹5,000

sales_amount = ₹10,000
```

20. Step 7 – Fact Payments

Payments are loaded into:

```
fact_payments
```

The fact contains information such as:

```
payment_id
order_id
amount
```

```
payment_method
payment_status
```

This allows the business to analyze:

- Total payments
- Payment methods
- Payment status
- Payment trends
- Order/payment relationships

21. Step 8 – Warehouse Reconciliation

After loading the warehouse, we reconcile source and target.

The objective is to verify:

```
Source
↓
Staging
↓
Warehouse
```

did not lose or duplicate data.

Examples:

```
Source customers
=
Staging customers
```

and:

```
Source payments
=
Fact payments
```

Reconciliation provides an independent validation of the ETL process.

22. Data Quality vs Reconciliation

These are different checks.

Reconciliation

Answers:

Did we move the expected amount of data?

Example:

```
Source = 45 orders
Target = 45 orders

Result = PASS
```

Data Quality

Answers:

Is the data valid?

Example:

```
orders = 45

But:

customer_id = NULL
```

Reconciliation could still pass.

Data quality would fail.

Therefore we need both:

```
Reconciliation
+
Data Quality
```

23. Step 9 – Reporting Layer

Once the warehouse is validated, reporting views are created.

Examples:

```
vw_sales_report  
vw_monthly_sales  
vw_customer_sales  
vw_product_sales  
vw_payment_summary
```

These views hide complex warehouse joins from dashboard users.

Instead of the dashboard querying multiple tables:

```
fact_sales  
dim_customer  
dim_product  
dim_date
```

it can query:

```
vw_monthly_sales
```

24. Step 10 – Dashboard

The dashboard consumes the reporting views.

Typical metrics include:

```
Total Sales  
Total Orders  
Total Units  
Total Customers  
Total Payments
```

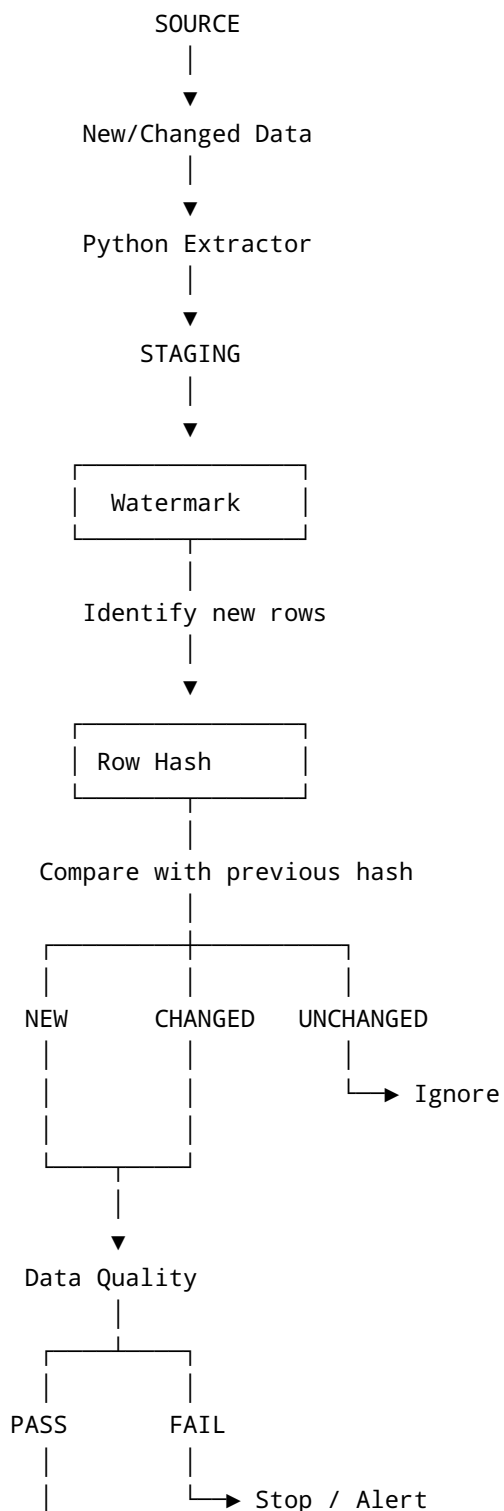
and visualizations such as:

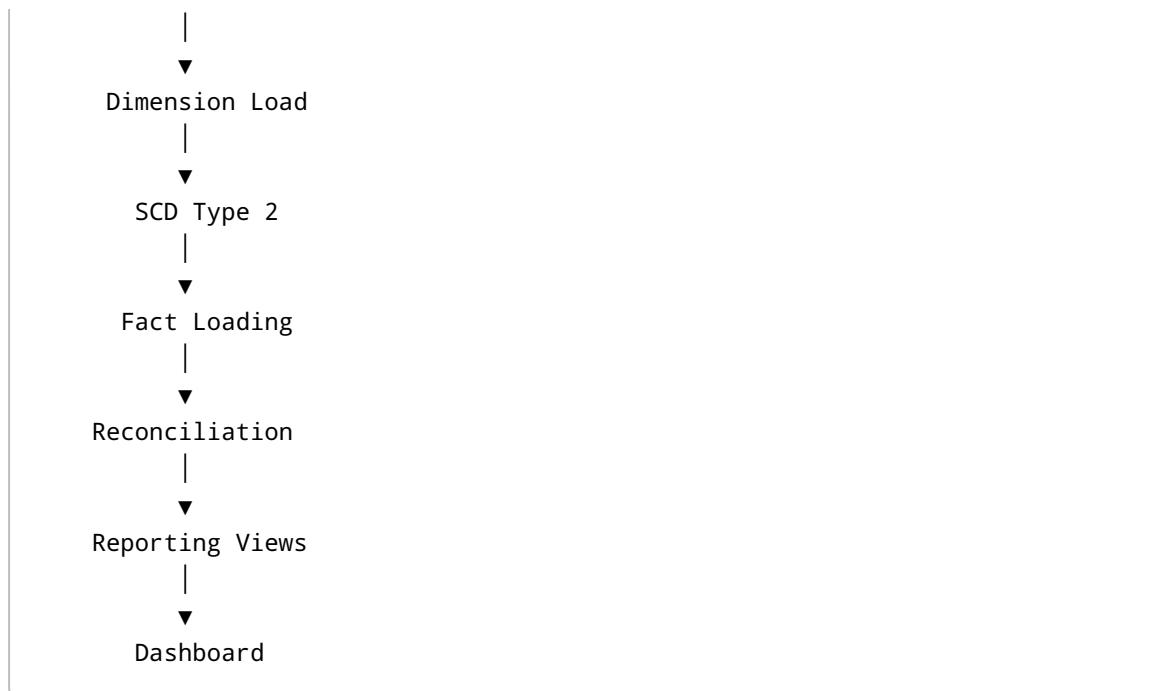
```
Monthly Sales  
Sales by Category  
Top Products  
Top Customers  
Payment Analysis  
Data Quality  
ETL Batch Status  
Reconciliation Status
```

The dashboard is therefore the final consumption layer.

25. Complete Incremental Workflow

The most important workflow to explain to the team is this:





26. Example – New Customer

Suppose the source receives:

```
customer_id = 23  
customer_name = New Customer  
city = Hyderabad
```

The next ETL run sees:

```
Watermark = 22
```

Therefore:

```
23 > 22
```

The record is extracted.

Classification:

```
NEW
```

The record passes data quality.

It is inserted into the warehouse.

The watermark becomes:

23

27. Example – Existing Customer Changed

Suppose:

```
customer_id = 22  
city = Hyderabad
```

changes to:

```
customer_id = 22  
city = Secunderabad
```

The watermark cannot detect this because:

```
22 is not greater than the watermark
```

The row hash detects the change:

```
Old Hash ≠ New Hash
```

Classification:

```
CHANGED
```

SCD Type 2 then creates:

```
Version 1  
Hyderabad  
is_current = 0  
  
Version 2  
Secunderabad  
is_current = 1
```

This preserves history.

28. Example – Unchanged Customer

If customer 22 has not changed:

Old Hash = New Hash

Classification:

UNCHANGED

No new dimension version is created.

This prevents unnecessary warehouse updates.

29. ETL Auditability

Every important operation should be traceable.

We can answer:

Which batch loaded the data?

etl_batch_control

What was the last processed record?

etl_watermark

Did the source record change?

etl_record_hash

Did the data-quality checks pass?

Great Expectations results

Did source and target reconcile?

Reconciliation results

This provides end-to-end ETL observability.

30. Failure Handling

If extraction fails:

Batch → FAILED

If data quality fails:

DQ → FAIL
Warehouse load → STOP

If warehouse loading fails:

Batch → FAILED

The failed batch remains recorded for investigation.

This is preferable to silently continuing with partially processed data.

31. Why This Architecture Is Reusable

The ETL framework is configuration-driven.

Instead of writing separate extraction logic for every table:

```
extract_customers()  
extract_products()  
extract_orders()  
...
```

the framework uses table configuration:

TABLE_CONFIG

and generic extraction/loading functions.

Therefore adding another table mainly requires:

1. Add the table to configuration.
2. Create its staging table.
3. Define its validation rules.
4. Define its warehouse mapping.

The generic framework can then process it.

32. Operational Run Sequence

For a normal production-style run:

```
1. Start ETL batch
  ↓
2. Extract incremental data
  ↓
3. Load staging
  ↓
4. Run data-quality checks
  ↓
5. Perform change detection
  ↓
6. Load dimensions
  ↓
7. Apply SCD Type 2
  ↓
8. Load fact tables
  ↓
9. Reconcile source vs target
  ↓
10. Update ETL metadata
  ↓
11. Refresh/report through views
  ↓
12. Dashboard consumes latest data
```

33. Initial Load vs Incremental Load

Area	Initial Load	Incremental Load
Extraction	All records	New/changed records

Area	Initial Load	Incremental Load
Watermark	Initialize	Advance
Hash	Create baseline	Compare
Staging	Full dataset	Delta
Dimensions	Initial population	Insert/update
SCD2	Initial version	New historical version
Facts	Initial population	New/changed transactions
Reconciliation	Full	Incremental/current
Dashboard	Initial data	Updated data

34. Example Business Scenario

Suppose the warehouse currently has:

```
Orders:      45
Order Items: 110
Payments:    38
```

A new laptop order worth ₹10,000 is created.

The next pipeline run:

```
Source
↓
New order detected
↓
Staging
↓
Data Quality PASS
↓
Fact Sales
↓
₹10,000 added to sales
↓
Fact Payments
↓
₹10,000 payment added
↓
Reporting View
```

↓
Dashboard

The dashboard now reflects the new transaction.

This demonstrates the complete end-to-end flow.

35. Technology Stack

```
Source / Target
  MySQL

ETL
  Python
  Pandas
  SQLAlchemy

Data Quality
  Great Expectations

Testing
  Pytest

Warehouse
  Star Schema
  SCD Type 2

Reporting
  MySQL Views

Dashboard
  Streamlit

Environment
  Python Virtual Environment
  Git
```

36. Key Design Principles

The solution follows these principles:

Separation of concerns

Extraction, transformation, loading, validation and reporting are separated.

Configuration driven

Table-specific behavior is maintained through configuration rather than duplicated code.

Incremental processing

Only new or changed records are processed after the initial load.

Idempotency

Repeated execution should not create duplicate warehouse records.

Auditability

Every batch and processing state is tracked.

Data quality

Bad data should be detected before it reaches analytical tables.

Historical tracking

SCD Type 2 preserves changes to important dimensions.

Reconciliation

Source and target are independently compared.

Testability

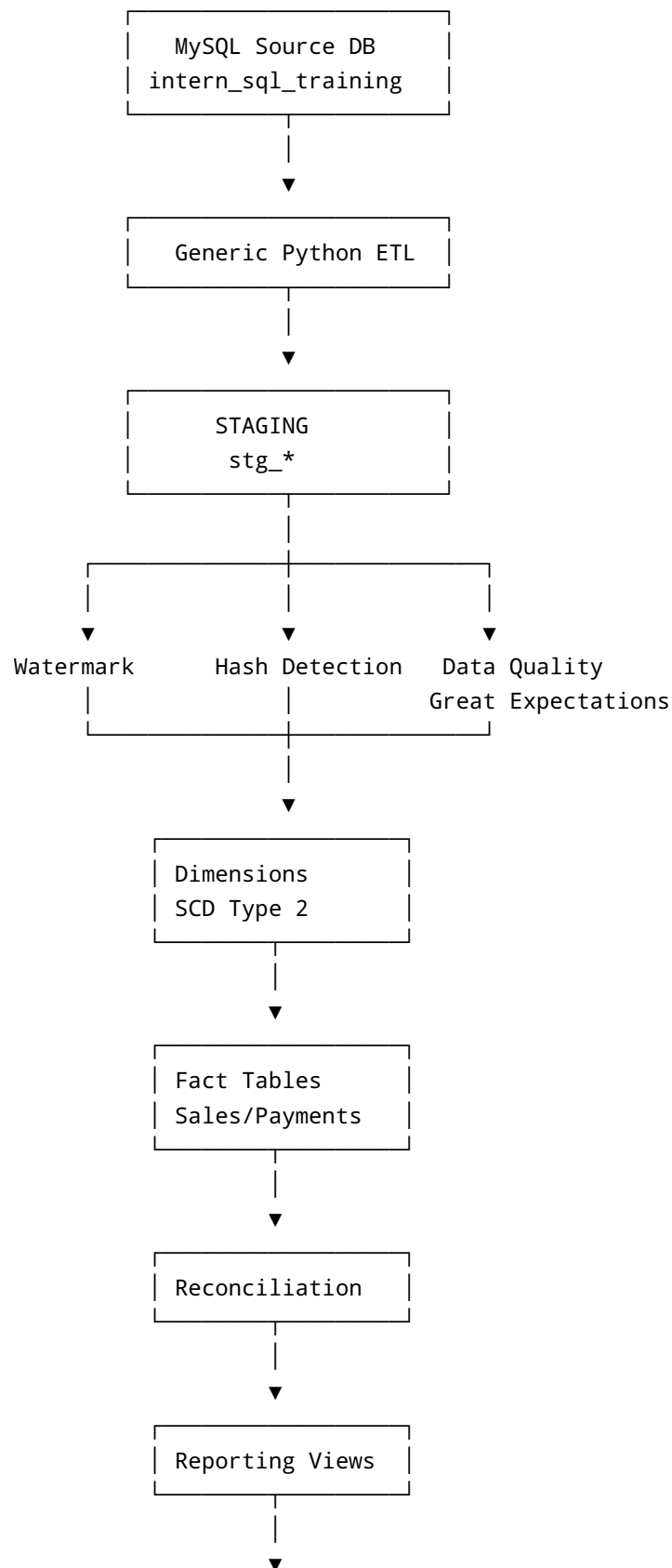
The pipeline contains automated tests for connections, ETL behavior, dimensions, facts and reconciliation.

37. One-Minute Explanation for the Team

The simplest way to explain the project is:

"We have an operational MySQL source and a separate retail data warehouse. The first load extracts all source data into staging. Every ETL run gets a batch ID for auditability. After the initial load, we use watermarks to identify new records. Because a watermark cannot detect updates to existing records, we also maintain row hashes and compare the current source hash with the previously processed hash to classify records as NEW, CHANGED or UNCHANGED. Data then passes through Great Expectations data-quality checks. Valid data is loaded into SCD Type 2 dimensions and fact tables such as sales and payments. Finally, we reconcile the warehouse against the source and expose reporting views to the dashboard. This gives us an end-to-end, incremental, auditable and quality-controlled ETL pipeline."

38. Architecture Summary



Dashboard

Final takeaway

The key message to the team is that this is **not simply a Python script that copies MySQL tables**.

It is a complete data pipeline with:

Extraction → Staging → Incremental Processing → Change Detection → Data Quality → SCD Type 2 → Fact Loading → Reconciliation → Reporting → Dashboard

with **batch control, watermarks, hashes, automated tests and data-quality gates** providing the operational controls around the pipeline.